Compilers: Every Method, One Practice Question, One Gotcha

29 methods for the midterm, each as a method box, a new practice question and the trap that costs points; answers at the end

Compilers

2026-10-31

How to use it. Read the method box, do the practice question on paper, then read the gotcha before checking the answer at the back (answer n belongs to method n). The tag on the right of each title says where the method was used in the mock midterm; "course" means it is in the lectures but the mock did not test it. Every answer was checked with `check_lex.py` and `check_ll1.py`. Conventions are the textbook's: Thompson's concatenation merges states (convention B), minimisation by table filling, stack top on the right, productions numbered top to bottom.

Contents

1 Which phase catches the error? [course] **3**

2 Writing a regular expression for a token [mock 1.7(b), 2.3(a)] **3**

3 Thompson's construction: regex to NFA [course] **3**

4 Simulating an NFA: ε-closure and move [course] **4**

5 Subset construction: NFA to DFA [mock 1.7(a)] **4**

6 DFA minimisation by table filling [course] **5**

7 State elimination: DFA to regex [course] **5**

8 Brzozowski derivatives [course] **6**

9 Designing a DFA directly from a description [mock 2.3(b)] **6**

10 Proving a language is not regular, and DFA lower bounds [mock 1.2, 2.3(c)] **6**

11 The combined scanner DFA and rule priority [mock 2.2, 2.4] **7**

12 Longest-match scanning with backup, and counting reads [mock 2.1] **8**

13 Lexer states for comments and strings [mock 2.3(c)] **9**

14 Writing a context-free grammar for a language [mock 1.2(b)] **9**

15 Leftmost and rightmost derivations, parse trees [mock 1.3] **10**

16 Ambiguity, precedence and associativity [mock 1.4(a)–(c)] **10**

17 Recursive descent: code from grammar, grammar from code [mock 4.1, 4.2(b), 4.5] **11**

18 Nullable nonterminals [mock 1.6(a), 5(a)] **12**

19 FIRST sets [mock 1.5(a), 3.2, 5(a)] **12**

20 FOLLOW sets [mock 1.5(a), 3.2, 5(a)] **13**

21 Building the LL(1) table and finding conflicts [mock 1.5(b), 3.3, 5(a)] **13**

22 The LL(1) stack trace [mock 4.3, 5(b)] **13**

23 Where is a syntax error caught? [mock 4.4] **14**

24 Removing immediate left recursion [mock 1.4(d), 3.1(b)] **14**

25 Removing indirect left recursion [course] **15**

26 Left factoring [course] **16**

27 Eliminating ε-productions [mock 1.6] **16**

28 Deciding "is this grammar LL(1)?" [mock 3.1(a), 5(a)] **17**

29 The Chomsky classification of a grammar or language [mock 1.1] **17**

Part A. Lexical analysis

1 Which phase catches the error? [course]

Method

Ask four questions in order. (1) **Lexical**: can the characters be cut into tokens by longest match? A character that starts no token, or an unterminated string or comment, is lexical. (2) **Syntax**: is the *token sequence* allowed by the grammar? (3) **Semantic**: does it break a rule the grammar cannot express (types, declarations, number of arguments)? (4) Otherwise it compiles; a wrong result is a run-time or logic error.

Practice

Classify each C fragment: (a) `x = 5 @ 2;` (b) `y = x + ;` (c) `int 3x = 5;` (d) `int a; a = "hi";` (e) `undeclared = 1;` (f) `s = "abc;` (g) `if (x) else y = 1;`

Gotcha

`3x` is *not* a lexical error: longest match cuts it into INT(3) ID(x), both legal tokens. The error appears only in the parser, which does not accept a number where a name is expected. A lexer never knows about grammar.

2 Writing a regular expression for a token [mock 1.7(b), 2.3(a)]

Method

(1) Cut the lexeme into parts; for each decide mandatory, optional (`?`), or repeated (`*`, `+`). (2) Write one sub-expression per part; union for alternatives. (3) Check each must-match example and each must-not example, naming the part that rejects it. Precedence: star $>$ concatenation $>$ union.

Practice

Over the alphabet {", \, x} (x = any other character), write a regex for string literals: a ", then characters where \ escapes the next character (any of the three), then a closing ". Must match: `""`, `"x\"x"`, `"\\"`. Must not match: `"x`, `"\"`, `"x"x"`.

Gotcha

Forgetting what a sub-expression must *not* contain. "Anything between quotes" written as " $(x \mid " \mid \backslash)^*$ " accepts `"x"x"`, and by longest match it would also swallow two strings and the code between them. The body must exclude an unescaped ".

3 Thompson's construction: regex to NFA [course]

Method

Build inductively; every fragment has one start and one final state. Symbol a: $i \xrightarrow{a} f$. **Concatenation** *st* **(convention B)**: the final state of $N(s)$ *is* the start of $N(t)$, no state added.

Union $s \mid t$: new i with ε to both starts, both finals ε to a new f. Star s^*: new i, f; ε edges $i \to$ start of s, final of $s \to$ start of s, final of $s \to f$, $i \to f$. $s^+ = ss^*$, $s? = s \mid \varepsilon$. State count: $2(\#\text{operators} + \#\text{operands}) - 3 \times \#\text{concatenations}$.

Practice

Build the convention-B NFA for $a(b \mid c)^*$, number its states, list its edges, and check the count with the formula. How many states would convention A (ε-edge per concatenation) give?

Gotcha

Two conventions exist and give different counts; say which one you use. The star needs *both* the back edge (repeat) and the $i \to f$ bypass (zero times); forgetting the bypass makes it s^+.

4 Simulating an NFA: ε-closure and move [course]

Method

Current set $S_0 = \varepsilon\text{-closure}(\{\text{start}\})$. For each input symbol c: $S \leftarrow \varepsilon\text{-closure}(\text{move}(S, c))$, where move follows only c-edges and the closure adds everything reachable by ε-edges alone. Accept iff the final set contains an accepting state. An empty set means reject immediately.

Practice

On your NFA for $a(b \mid c)^*$, simulate `acb`: give the set after each symbol. Is it accepted? Then simulate `ca`.

Gotcha

Take the closure *after every move*, and also of the start state before reading anything. Forgetting the initial closure makes ε-paths from the start invisible.

5 Subset construction: NFA to DFA [mock 1.7(a)]

Method

Start state $A = \varepsilon\text{-closure}(\{s_0\})$. For each unprocessed set S and symbol c: $T = \varepsilon\text{-closure}(\text{move}(S, c))$; name it if new. Stop when no new set appears. A set is accepting iff it contains an NFA accepting state. The empty set is the dead state, usually not drawn.

Practice

NFA over $\{a, b\}$: states $0, 1, 2$, start 0, accepting 2; edges $0 \xrightarrow{a,b} 0$, $0 \xrightarrow{a} 1$, $1 \xrightarrow{a,b} 2$ ("the second symbol from the end is a"). Build the DFA. How many states, and why can no DFA have fewer?

Gotcha

A set is a set: $\{0, 1\}$ and $\{1, 0\}$ are the same state. And a DFA state is accepting if it contains *any* accepting NFA state, not all of them.

6 DFA minimisation by table filling [course]

Method

(1) List all pairs (add a dead state if the DFA is incomplete). (2) Mark every accepting/non-accepting pair. (3) Sweep: mark (p, q) if some symbol sends it to an already marked pair; repeat full sweeps until none marks anything. (4) Unmarked pairs are equivalent: merge them and redirect every edge into a merged state.

Practice

The subset construction for $(a \mid b)^*ab$ gives: A (start): $a \to B$, $b \to D$; B: $a \to B$, $b \to C$; C (accepting): $a \to B$, $b \to D$; D: $a \to B$, $b \to D$. Minimise it, giving the sweep in which each pair is marked and a witness string.

Gotcha

A sweep may only use marks from *earlier* sweeps if you want to report "when" a pair was marked; and you must do one more full sweep that marks nothing before stopping. Two states with identical rows are equivalent immediately, but equivalent states need not have identical rows.

7 State elimination: DFA to regex [course]

Method

(1) If the start has incoming edges, add a new start with ε to it; add a new final f with ε from every accepting state. (2) Remove the other states one at a time: for each path $p \xrightarrow{A} q \xrightarrow{B} r$ through the removed q with self-loop L, add $p \xrightarrow{AL^*B} r$, union with any existing $p \to r$ edge. (3) Read the label of the last edge (with X^*Y if a loop X remains on the start).

Practice

DFA over $\{a, b\}$: state 0 (start, accepting), state 1; $0 \xrightarrow{b} 0$, $0 \xrightarrow{a} 1$, $1 \xrightarrow{b} 1$, $1 \xrightarrow{a} 0$. Give a regex by state elimination. Which language is it?

Gotcha

Forgetting the new start state when the old one has incoming edges: you would then read the start's loop wrongly. Every path through q gets a new edge, including $p = r$ (that creates a self-loop).

8 Brzozowski derivatives [course]

Method

$\partial_a R$ = what is left of R after reading a. Rules: $\partial_a a = \varepsilon$, $\partial_a b = \emptyset$, $\partial_a \varepsilon = \partial_a \emptyset = \emptyset$, $\partial_a(R \mid S) = \partial_a R \mid \partial_a S$, $\partial_a(R^*) = (\partial_a R)R^*$, $\partial_a(RS) = (\partial_a R)S \mid \nu(R)\,\partial_a S$, where $\nu(R) = \varepsilon$ if R accepts ε and $\emptyset$ otherwise. A string w is accepted iff $\nu(\partial_w R) = \varepsilon$.

Practice

For $R = (ab)^*$, compute $\partial_a R$, $\partial_b R$ and the derivatives of those, until no new expression appears. Use them to decide `abab` and `aba`. Draw the DFA whose states are the expressions.

Gotcha

The concatenation rule has a second term: when R is nullable, a may be read by S. Leaving out $\nu(R)\partial_a S$ breaks every example with an optional prefix.

9 Designing a DFA directly from a description [mock 2.3(b)]

Method

Decide what the automaton must *remember* about the prefix read so far (a remainder, how much of a pattern has been matched, inside/outside a comment). One state per possible memory value. Write the transition for every state and symbol, then mark the accepting memories. Test on two accepted and two rejected strings.

Practice

Build a DFA over $\{0, 1\}$ accepting binary numbers (most significant bit first) divisible by 3. Check it on `110` and `111`.

Gotcha

Reading one more bit doubles the number: the new remainder is $(2r+b)$ mod 3, not $(r+b)$ mod 3. For comments, a final state with outgoing edges would accept text after the first `*/`.

10 Proving a language is not regular, and DFA lower bounds [mock 1.2, 2.3(c)]

Method

Distinguishable strings: u, v are distinguishable if some suffix z puts exactly one of uz, vz in L; they must reach different DFA states. Infinitely many pairwise distinguishable strings: not regular. k pairwise distinguishable strings: every DFA has at least k states. **Pumping**: pick $w \in L$ with $|w| \geq p$ whose first p symbols are equal, so the pumped part lies inside them; pump and leave L.

Practice

(a) Prove $L = \{a^n b^m : m \geq n\}$ is not regular. (b) Is $\{a^n b^m : n, m \geq 0\}$ regular? (c) Prove that every DFA for "the third symbol from the end is a" over $\{a, b\}$ has at least 8 states.

Gotcha

A language written with exponents is not automatically non-regular: $a^n b^m$ with independent n, m is $a^* b^*$. Non-regularity needs a *dependence* the automaton would have to count without bound.

11 The combined scanner DFA and rule priority [mock 2.2, 2.4]

Method

Build one automaton for all rules (one NFA per rule, joined by ε from a new start, then subsets, or directly by hand with character classes). Label each accepting state with the **first-listed** rule it accepts. A rule that labels no state can never fire.

Practice

Rules in this order: (1) `=` ASSIGN, (2) `==` EQ, (3) `=>` ARROW, (4) `[a-z]+` ID, (5) `in` IN. Draw the DFA with classes `=`, `>`, `i`, `n`, L (other letters). Label the states. What does `in=>inx==` give? Fix the specification.

Gotcha

Priority only decides *ties in length*. A keyword listed after the identifier rule is dead (flex warns "rule cannot be matched"), but a keyword listed first still loses to the identifier on `inx`, because the identifier match is longer.

12 Longest-match scanning with backup, and counting reads [mock 2.1]

Method

From the start state, read while a transition exists; remember the last accepting state and its position. When stuck or at end of input, back up to that position, emit its token, restart there. No accepting state seen: error on one character, skip it. Characters read past the emitted token are re-read later (pushback).

Practice

Rules in order: (1) `a`, (2) `abb`, (3) `a*b+`. Scan `abbbaab`: give each token, its rule, the pushback, and the total number of characters read. Then: with rules `a` and `a*b`, how many characters are read on `aaaa`?

Gotcha

Do not stop at the first accepting state: `abb` matches rule 2, but the scanner continues to `abbb`. And a scanner that reached a non-accepting state (`7.` in the mock) backs up to the last *accepting* one, not to the previous character.

13 Lexer states for comments and strings [mock 2.3(c)]

Method

When a token is not regular or needs context (nested comments, strings with escapes, heredocs), switch the lexer into a **state** (flex: start condition) with its own rules, and keep extra memory in a variable (a depth counter). Return to the initial state when the construct closes; handle end of file inside it.

Practice

Write flex-style rules for nested `/* */` comments with a depth counter. Trace the depth on `/* a /* b */ c */ x` and say which token is returned after the comment.

Gotcha

Forgetting the end-of-file rule inside the comment state: an unterminated comment then silently eats the rest of the program instead of producing a lexical error.

Part B. Grammars and top-down parsing

14 Writing a context-free grammar for a language [mock 1.2(b)]

Method

Find the matching structure: things that must balance are generated *together* by one recursive rule ($S \to aSb$ adds one of each). Independent parts get their own nonterminal. Check by generating the smallest strings, and argue that nothing outside L is generated.

Practice

Give CFGs for (a) $\{a^n b^m c^{n+m} : n, m \geq 0\}$; (b) $\{a^i b^j : i \leq j \leq 2i\}$; (c) strings over $\{a, b\}$ with as many a's as b's.

Gotcha

Generating a superset: $S \to aS \mid Sb \mid \varepsilon$ produces a^*b^*, with no link between the counts. The balancing letters must be produced in the *same* step.

15 Leftmost and rightmost derivations, parse trees [mock 1.3]

Method

Leftmost: at each step expand the leftmost nonterminal; rightmost: the rightmost. Work backwards from the sentence to see which production each part needs. The tree is the same for both; only the order of expansion differs. Write every step; skipping steps loses marks.

Practice

Postfix grammar $S \to S\,S\,\texttt{+} \mid S\,S\,\texttt{*} \mid a$, sentence `aa+a*`. Give a leftmost and a rightmost derivation and the parse tree.

Gotcha

In a rightmost derivation, the *first* steps fix the right end of the sentence. And one parse tree has exactly one leftmost derivation: if you find two leftmost derivations, you found ambiguity.

16 Ambiguity, precedence and associativity [mock 1.4(a)–(c)]

Method

Prove ambiguity with *one* sentence and *two* parse trees. Remove it for operators by layering: one nonterminal per precedence level (loosest at the top), left recursion for left-associative operators, right recursion for right-associative ones; parentheses at the bottom restart from the top.

Practice

$E \to E{+}E \mid E{*}E \mid E\texttt{\^{}}E \mid (E) \mid \mathsf{id}$. (a) Show ambiguity on id + id * id. (b) Write an unambiguous grammar with ^ highest and right-associative, * next, + lowest, both left-associative. (c) Draw the tree of id ^ id ^ id in your grammar.

Gotcha

Two different *derivations* are not proof of ambiguity (a leftmost and a rightmost derivation of the same tree always exist). It takes two different *trees*, or two different leftmost derivations.

17 Recursive descent: code from grammar, grammar from code

[mock 4.1, 4.2(b), 4.5]

Method

One procedure per nonterminal; an `if` per production, tested on its selection set; inside, `match(t)` for terminals and calls for nonterminals, left to right; an ε-production is "return"; no matching branch: `error()`. Reading code backwards: each branch is a production and its condition is the selection set.

Practice

Grammar $S \to (L) \mid a$, $L \to S\,L'$, $L' \to \texttt{,}\,S\,L' \mid \varepsilon$. Write `S()`, `L()`, `Lp()`. List the calls made on `(a,a)`.

Gotcha

A left-recursive rule $A \to A\alpha$ becomes `A(){ A(); ...}`: infinite recursion, no token consumed. And an ε branch written as "otherwise return" accepts the same language but reports errors later than the table would.

18 Nullable nonterminals

[mock 1.6(a), 5(a)]

Method

Mark A if it has $A \to \varepsilon$. Then repeatedly mark A if some production of A has a right side made only of marked nonterminals. Stop when a full pass marks nothing.

Practice

$E \to A\,e \mid \varepsilon$, $A \to B\,C$, $B \to \varepsilon \mid b$, $C \to D\,D$, $D \to d \mid \varepsilon$. Which nonterminals are nullable, and in which pass is each found?

Gotcha

A right side containing a terminal is never nullable ($E \to Ae$ does not make E nullable; $E \to \varepsilon$ does). Nullability propagates through chains, so one pass is not enough.

19 FIRST sets

[mock 1.5(a), 3.2, 5(a)]

Method

$\text{FIRST}(t) = \{t\}$. For $X \to Y_1 \cdots Y_k$: add $\text{FIRST}(Y_1) \setminus \varepsilon$; while Y_i is nullable continue with Y_{i+1}; stop at the first non-nullable symbol; if all are nullable add ε. Iterate to a fixed point.

Practice

$S \to A\,B\,C$, $A \to a\,A \mid \varepsilon$, $B \to b\,B \mid C$, $C \to c \mid \varepsilon$. Compute FIRST of every nonterminal.

Gotcha

$\varepsilon \in \text{FIRST}(X)$ only if *every* symbol of some right side can vanish. In the expression grammar $T \to FT'$, T' is nullable but F is not, so $\varepsilon \notin \text{FIRST}(T)$.

20 FOLLOW sets [mock 1.5(a), 3.2, 5(a)]

Method

\$ ∈ FOLLOW(start). For every *occurrence* $B \to \alpha A \beta$: add FIRST(β) \ ε to FOLLOW(A); if β is nullable or empty, add FOLLOW(B). Iterate to a fixed point.

Practice

Same grammar as §19. Compute FOLLOW of every nonterminal, naming the occurrence that contributes each element.

Gotcha

FOLLOW never contains ε. And a self-reference such as $B \to bB$ ("add FOLLOW(B) to FOLLOW(B)") adds nothing; do not loop on it.

21 Building the LL(1) table and finding conflicts [mock 1.5(b), 3.3, 5(a)]

Method

Number the productions. For $A \to \alpha$: put it in $M[A, a]$ for each $a \in$ FIRST(α) \ ε; if α is nullable, also in $M[A, b]$ for each $b \in$ FOLLOW(A). A cell with two entries is a conflict: FIRST/FIRST (two alternatives start alike) or FIRST/FOLLOW (a nullable alternative meets another's FIRST).

Practice

Same grammar as §19–20. Build the table. Is the grammar LL(1)? If not, name the cell, the two productions, the kind of conflict, and a sentence that shows what goes wrong.

Gotcha

Use FIRST of the *right side* α, not FIRST of A: for $B \to C$ only FIRST(C) counts, never b. And an ε-production goes under FOLLOW, never under "ε" (there is no ε column).

22 The LL(1) stack trace [mock 4.3, 5(b)]

Method

Stack \$ S (top right), input followed by \$. Top terminal: match and advance, or error. Top nonterminal X, lookahead a: replace X by the right side of $M[X, a]$ pushed in *reverse* (first symbol on top); ε: just pop; empty cell: error. Accept when both are \$.

Practice

Grammar of §17 with productions 1 $S \to (L)$, 2 $S \to a$, 3 $L \to SL'$, 4 $L' \to$ `,`SL', 5 $L' \to \varepsilon$. Build the table and trace `(a,(a))`.

Gotcha

Only a *match* advances the input. Writing the input one symbol shorter after an expansion is the most common trace error, followed by pushing the right side in reading order.

23 Where is a syntax error caught? [mock 4.4]

Method

Table parser: at the first empty cell $M[X, a]$, or at a terminal on top that differs from the lookahead. Recursive descent: in the procedure whose tests fail, which can be *later* if a procedure has a default "return" branch or checks nothing before calling another procedure.

Practice

Grammar of §17. Input `(a a)`. Where does the table parser stop? Where does a recursive-descent parser stop if `Lp()` is written as "if lookahead is `,` then ..., otherwise return"?

Gotcha

Both parsers reject the same inputs; they differ only in *where* they notice. The table, with exact selection sets, never notices later than the code.

24 Removing immediate left recursion [mock 1.4(d), 3.1(b)]

Method

$A \to A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n$ becomes $A \to \beta_1 A' \mid \cdots \mid \beta_n A'$, $A' \to \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon$. For a list rule $L \to L\,X \mid \varepsilon$ with no new nonterminal allowed: $L \to X\,L \mid \varepsilon$.

Practice

Remove the left recursion from $R \to R\,a \mid R\,b\,c \mid d \mid e\,f$.

Gotcha

Every non-recursive alternative β_i gets the A' suffix, and A' needs its ε-alternative. Dropping either changes the language.

25 Removing indirect left recursion [course]

Method

Order the nonterminals $A_1, \ldots, A_n$. For $i = 1..n$: for every $j < i$, replace each $A_i \to A_j\gamma$ by $A_i \to \delta\gamma$ for every production $A_j \to \delta$; then remove the immediate left recursion of A_i. Afterwards, check LL(1) separately.

Practice

$A \to B\,a \mid c$, $B \to A\,b \mid d$. (a) Remove the left recursion with the order A, B; is the result LL(1)? (b) Redo it with the order B, A. Which do you hand in?

Gotcha

Removing left recursion is necessary, not sufficient: the result can still have conflicts, and the order of the nonterminals changes the result. Always rebuild the table.

26 Left factoring [course]

Method

For alternatives with a common prefix, $A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma$ becomes $A \to \alpha A' \mid \gamma$, $A' \to \beta_1 \mid \beta_2$. Take the *longest* common prefix and repeat until no two alternatives of any nonterminal share a first symbol.

Practice

(a) Left factor $A \to a\,b\,c \mid a\,b\,d \mid a\,e$. (b) Left factor $S \to \mathsf{i}\,E\,\mathsf{t}\,S \mid \mathsf{i}\,E\,\mathsf{t}\,S\,\mathsf{e}\,S \mid \mathsf{a}$, $E \to \mathsf{b}$, and decide whether the result is LL(1).

Gotcha

Left factoring cannot fix ambiguity. The dangling else stays a conflict in $M[S', \mathsf{e}]$ after factoring; the parser resolves it by keeping $S' \to \mathsf{e}\,S$ (each else goes to the nearest if).

27 Eliminating ε-productions [mock 1.6]

Method

(1) Find the nullable nonterminals. (2) For each production, add every version with any subset of its nullable occurrences deleted, except an empty right side. (3) Delete the ε-productions. (4) If the start symbol was nullable, add $S' \to S \mid \varepsilon$ with a new start S'.

Practice

$S \to A\,B \mid a$, $A \to a\,A \mid \varepsilon$, $B \to b\,B \mid \varepsilon$.

Gotcha

With k nullable occurrences in one right side you need all 2^k versions (minus the empty one): $(S)S$ in the mock gives four. Forgetting the new start loses ε from the language.

28 Deciding "is this grammar LL(1)?" [mock 3.1(a), 5(a)]

Method

In order: ambiguous? left-recursive? common prefixes? For each nonterminal, are the FIRST sets of its alternatives disjoint, and, for a nullable alternative, is FOLLOW disjoint from the other alternatives' FIRST? If all pass, it is LL(1). Justify a "no" with the cell and the two productions in it.

Practice

LL(1) or not, with a one-line reason: (i) $S \to aSb \mid \varepsilon$; (ii) $S \to aS \mid a$; (iii) $S \to Aa \mid b$, $A \to b \mid \varepsilon$; (iv) $S \to Sa \mid b$; (v) $S \to aA \mid b$, $A \to cA \mid \varepsilon$; (vi) $E \to E + E \mid \mathsf{id}$.

Gotcha

"Not LL(1)" is a property of the grammar, not of the language: (ii) and (iv) become LL(1) after factoring or removing left recursion. Only an inherently ambiguous language has no LL(1) grammar at all.

29 The Chomsky classification of a grammar or language [mock 1.1]

Method

Look at the production shapes: $A \to aB \mid a$ (right-linear) is type 3; one nonterminal on the left with any right side is type 2; $|\alpha| \leq |\beta|$ is type 1. For a *language*, the type is the most restrictive grammar that exists for it: prove non-regularity (§10) to rule out type 3.

Practice

Classify the grammar and the language: (a) $S \to aS \mid bS \mid a$; (b) $S \to aSb \mid \varepsilon$; (c) $S \to aS \mid Sb \mid \varepsilon$.

Gotcha

A grammar that is not right-linear can still describe a regular language, as (c) does: classify the language by the simplest grammar that *could* describe it.

Answers

Answer: 1. Which phase

(a) lexical (`@` starts no token); (b) syntax; (c) syntax (`INT ID` where a declarator is expected); (d) semantic (type); (e) semantic (undeclared); (f) lexical (unterminated string); (g) syntax.

Answer: 2. String literal regex

" $(x \mid \backslash(x \mid " \mid \backslash))^*$ ". `"x\"x"` matches: x, then the escape `\"`, then x. `"x` fails: no closing quote. `"\"` fails: `\"` is an escape, so the string is never closed. `"x"x"` fails: the body cannot contain a bare `"`.

Answer: 3. Thompson for $a(b \mid c)^*$

States 0–8, start 0, final 2. $0 \xrightarrow{a} 1$ (state 1 is both the end of $N(a)$ and the start of the star). Star: $1 \xrightarrow{\varepsilon} 3$, $1 \xrightarrow{\varepsilon} 2$ (bypass), $4 \xrightarrow{\varepsilon} 3$ (repeat), $4 \xrightarrow{\varepsilon} 2$. Union inside: $3 \xrightarrow{\varepsilon} 5$, $3 \xrightarrow{\varepsilon} 7$, $5 \xrightarrow{b} 6$, $7 \xrightarrow{c} 8$, $6 \xrightarrow{\varepsilon} 4$, $8 \xrightarrow{\varepsilon} 4$. Count: operands 3, operators 3 (concatenation, union, star): $2 \times 6 - 3 \times 1 = 9$. Convention A keeps both states and adds one ε-edge: $9 + 1 = 10$ states.

Answer: 4. Simulating `acb`

Start: ε-closure$\{0\} = \{0\}$. After `a`: ε-closure$\{1\} = \{1, 2, 3, 5, 7\}$. After `c`: ε-closure$\{8\} = \{8, 4, 3, 5, 7, 2\}$. After `b`: ε-closure$\{6\} = \{6, 4, 3, 5, 7, 2\}$, contains 2: **accepted**. On `ca`: move$(\{0\}, c) = \emptyset$: rejected at the first symbol.

Answer: 5. Subset construction

$A = \{0\}$: $a \to B = \{0, 1\}$, $b \to A$. B: $a \to C = \{0, 1, 2\}$, $b \to D = \{0, 2\}$. C: $a \to C$, $b \to D$. D: $a \to B$, $b \to A$. Accepting: C, D. Four states, which is the minimum: the four strings aa, ab, ba, bb are pairwise distinguishable (the DFA must remember the last two symbols, 2^2 possibilities).

Answer: 6. Minimisation

Pairs: 6. Rule 1: (A, C), (B, C), (D, C), witness ε. Sweep 1: (A, B) on b goes to (D, C), marked, witness b; (B, D) on b goes to (C, D), witness b. (A, D): on a (B, B), on b (D, D): never marked. Sweep 2 marks nothing. Merge $A \equiv D$: three states $\{A, D\}$ (start), B, C (accepting); $\{A, D\}$: $a \to B$, $b \to \{A, D\}$; B: $a \to B$, $b \to C$; C: $a \to B$, $b \to \{A, D\}$. Meaning: "nothing", "seen a", "seen ab".

Answer: 7. State elimination

0 has incoming edges: add $s \xrightarrow{\varepsilon} 0$ and $0 \xrightarrow{\varepsilon} f$. Remove 1 (loop b): path $0 \xrightarrow{a} 1 \xrightarrow{a} 0$ gives the loop ab^*a on 0, joined with b: loop $b \mid ab^*a$. Remove 0: $s \xrightarrow{(b \mid ab^*a)^*} f$. Language: strings with an **even number of a's**.

Answer: 8. Derivatives of $(ab)^*$

$\partial_a(ab)^* = (\partial_a(ab))(ab)^* = b(ab)^*$; $\partial_b(ab)^* = \emptyset$. $\partial_a(b(ab)^*) = \emptyset$ (b is not nullable); $\partial_b(b(ab)^*) = (ab)^*$. Expressions: $R = (ab)^*$ (nullable, accepting), $R_1 = b(ab)^*$, $\emptyset$ (dead). DFA: $R \xrightarrow{a} R_1$, $R_1 \xrightarrow{b} R$. `abab`: $R \to R_1 \to R \to R_1 \to R$, nullable: accepted. `aba`: ends in R_1, not nullable: rejected.

Answer: 9. Divisible by 3

States $0, 1, 2$ = remainder, start and accepting 0; $\delta(r, b) = (2r + b) \bmod 3$: $0 \xrightarrow{0} 0$, $0 \xrightarrow{1} 1$, $1 \xrightarrow{0} 2$, $1 \xrightarrow{1} 0$, $2 \xrightarrow{0} 1$, $2 \xrightarrow{1} 2$. `110` (6): $0 \to 1 \to 0 \to 0$, accepted. `111` (7): $0 \to 1 \to 0 \to 1$, rejected.

Answer: 10. Non-regularity and lower bounds

(a) a^i, a^j with $i < j$: the suffix b^i gives $a^i b^i \in L$ and $a^j b^i \notin L$. Infinitely many distinguishable strings: not regular. (b) Yes: a^*b^*, two states (plus dead). (c) Take the 8 strings of length 3. If $u \neq v$ first differ at position k (from the left), append $z = b^{k-1}$: that position becomes the third from the end, a in one and b in the other. 8 pairwise distinguishable strings: at least 8 states.

Answer: 11. Combined scanner DFA

States: 0 start; $0 \xrightarrow{=} 1$ ASSIGN; $1 \xrightarrow{=} 2$ EQ; $1 \xrightarrow{>} 3$ ARROW; $0 \xrightarrow{i} 4$ ID; $4 \xrightarrow{n} 5$, accepting ID and IN, labelled ID (listed first); $0 \xrightarrow{n,L} 6$, $4 \xrightarrow{i,L} 6$, $5 \xrightarrow{i,n,L} 6$, $6 \xrightarrow{i,n,L} 6$, all ID. IN labels no state: dead rule. `in=>inx==` gives ID(in) ARROW ID(inx) EQ. Fix: list `in` before the identifier rule; then state 5 is IN.

Answer: 12. Longest match

From position 0: `a` (rule 1), `ab` (3), `abb` (rules 2 and 3 tie, rule 2), `abbb` (3), then `a` cannot continue `a*b+`: stuck after reading 5 characters, back up 1. Token R3 `abbb`, pushback 1. Next: `a`, `aa`, `aab` (rule 3), end of input: token R3 `aab`. Total reads $5 + 3 = 8$. With `a` and `a*b` on `aaaa`: each token reads to the end hoping for `b`: $4 + 3 + 2 + 1 = 10$ reads for four tokens `a`.

Answer: 13. Nested comments

```
"/*"              { depth = 1; BEGIN(COMMENT); }
<COMMENT>"/*"     { depth++; }
<COMMENT>"*/"     { if (--depth == 0) BEGIN(INITIAL); }
<COMMENT>.|\n     { /* skip */ }
<COMMENT><<EOF>> { error("unterminated comment"); }
```

Depth: 1 at the first `/*`, 2 at the second, 1 after `b */`, 0 after `c */`: back to INITIAL; then ID(x).

Answer: 14. Grammars

(a) $S \to aSc \mid B$, $B \to bBc \mid \varepsilon$ (each a and each b is matched by its own c, b's inside). (b) $S \to aSb \mid aSbb \mid \varepsilon$ (each a comes with one or two b's). (c) $S \to aSbS \mid bSaS \mid \varepsilon$.

Answer: 15. Postfix derivations

Leftmost: $S \underset{lm}{\Rightarrow} SS* \underset{lm}{\Rightarrow} SS+S* \underset{lm}{\Rightarrow} aS+S* \underset{lm}{\Rightarrow} aa+S* \underset{lm}{\Rightarrow} aa+a*$.
Rightmost: $S \underset{rm}{\Rightarrow} SS* \underset{rm}{\Rightarrow} Sa* \underset{rm}{\Rightarrow} SS+a* \underset{rm}{\Rightarrow} Sa+a* \underset{rm}{\Rightarrow} aa+a*$.
Tree: root S with children S, S, `*`; the first child S has children $S(a)$, $S(a)$, `+`; the second is $S(a)$.

Answer: 16. Layered grammar

(a) Trees: $E(E(\mathsf{id}) + E(E(\mathsf{id}) * E(\mathsf{id})))$ and $E(E(E(\mathsf{id}) + E(\mathsf{id})) * E(\mathsf{id}))$. (b) $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to P\hat{}F \mid P$, $P \to (E) \mid \mathsf{id}$. (c) $F(P(\mathsf{id})\,\hat{}\,F(P(\mathsf{id})\,\hat{}\,F(P(\mathsf{id}))))$: nested to the right, $\mathsf{id}^{(\mathsf{id}^{\mathsf{id}})}$.

Answer: 17. Recursive descent for the list grammar

```
void S()  { if (la == '(') { match('('); L(); match(')'); }
            else if (la == 'a') match('a'); else error(); }
void L()  { if (la == '(' || la == 'a') { S(); Lp(); } else error(); }
void Lp() { if (la == ',') { match(','); S(); Lp(); }
            else if (la == ')') { /* eps: FOLLOW(L') = { ) } */ } else error(); }
```

Calls on `(a,a)`: `S, match((), L, S, match(a), Lp, match(,), S, match(a), Lp` (sees `)`, returns)`, match()`).

Answer: 18. Nullable

Pass 1: E ($E \to \varepsilon$), B ($B \to \varepsilon$), D ($D \to \varepsilon$). Pass 2: C ($C \to DD$), then A ($A \to BC$) in the same or the next pass depending on order. Pass 3 adds nothing. All five are nullable; $E \to Ae$ is *not* the reason E is nullable.

Answer: 19. FIRST

Nullable: C, then B (via $B \to C$), A, S. $\text{FIRST}(C) = \{c, \varepsilon\}$; $\text{FIRST}(B) = \{b\} \cup \text{FIRST}(C) = \{b, c, \varepsilon\}$; $\text{FIRST}(A) = \{a, \varepsilon\}$; $\text{FIRST}(S)$: scan ABC, all nullable: $\{a, b, c, \varepsilon\}$.

Answer: 20. FOLLOW

$\text{FOLLOW}(S) = \{\$\}$. $\text{FOLLOW}(A)$: in $S \to ABC$, $\text{FIRST}(BC) \setminus \varepsilon = \{b, c\}$, and BC nullable: $+\text{FOLLOW}(S)$: $\{b, c, \$\}$. $\text{FOLLOW}(B)$: in $S \to ABC$, $\text{FIRST}(C) \setminus \varepsilon = \{c\}$, C nullable: $+\{\$\}$; $B \to bB$ adds itself: $\{c, \$\}$. $\text{FOLLOW}(C)$: last in $S \to ABC$: $\{\$\}$; last in $B \to C$: $\text{FOLLOW}(B)$: $\{c, \$\}$.

Answer: 21. Table and conflict

Productions: 1 $S \to ABC$; 2 $A \to aA$; 3 $A \to \varepsilon$; 4 $B \to bB$; 5 $B \to C$; 6 $C \to c$; 7 $C \to \varepsilon$. Row S: 1 under $a, b, c, \$$. Row A: 2 under a; 3 under $b, c, \$$. Row B: 4 under b; 5 under $c, \$$. Row C: 6 under c; 7 under $c, \$$. **Conflict at** $M[C, c]$: 6 and 7, a FIRST/FOLLOW conflict ($c \in \text{FIRST}(c) \cap \text{FOLLOW}(C)$). Not LL(1); in fact ambiguous: in the sentence `c`, the `c` can come from $B \to C \to c$ or from the last C.

Answer: 22. Table and trace for `(a,(a))`

Table: $M[S,(] = 1$, $M[S,a] = 2$, $M[L,(] = M[L,a] = 3$, $M[L',,] = 4$, $M[L',)] = 5$ (FOLLOW$(L') = \{)\}$).

Stack	Input	Act.	Stack	Input	Act.
\$ S	`(a,(a))$`	1	\$ $L'\,S$	`(a))$`	1
\$) L (	`(a,(a))$`	match	\$ L') L (	`(a))$`	match
\$) L	`a,(a))$`	3	\$ L') L	`a))$`	3
\$) $L'\,S$	`a,(a))$`	2	\$ L') $L'\,S$	`a))$`	2
\$) $L'\,a$	`a,(a))$`	match	\$ L') $L'\,a$	`a))$`	match
\$) L'	`,(a))$`	4	\$ L') L'	`))$`	5
\$) $L'\,S$,	`,(a))$`	match	\$ L')	`))$`	match
			\$ L'	`)$`	5
			\$)	`)$`	match
			\$	`$`	accept

Answer: 23. Where the error is caught

Input `(a a)`. Table: after $S \to (L)$, match `(`, $L \to SL'$, $S \to a$, match `a`: top L', lookahead `a`, $M[L',a]$ is empty: error at L'. Code: `Lp()` sees `a`, not `,`, and returns; `S()` then calls `match(')')` on `a`: error in `match`, one step later.

Answer: 24. Immediate left recursion

α's: a, bc; β's: d, ef. $R \to d\,R' \mid e\,f\,R'$, $R' \to a\,R' \mid b\,c\,R' \mid \varepsilon$.

Answer: 25. Indirect left recursion

(a) Order A, B: A has no $A \to A\ldots$. For B, substitute A in $B \to Ab$: $B \to Bab \mid cb \mid d$; remove: $B \to cbB' \mid dB'$, $B' \to abB' \mid \varepsilon$. Not LL(1): FIRST$(Ba) = \{c,d\}$ meets FIRST(c) at $M[A,c]$, and FOLLOW$(B') = \{a\}$ meets FIRST(abB') at $M[B',a]$. (b) Order B, A: substitute B in $A \to Ba$: $A \to Aba \mid da \mid c$; remove: $A \to da\,A' \mid c\,A'$, $A' \to ba\,A' \mid \varepsilon$ (B is now unreachable). FOLLOW$(A') = \{\$\}$, disjoint from $\{b\}$: LL(1). Hand in (b): the language is $(c \mid da)(ba)^*$.

Answer: 26. Left factoring

(a) $A \to aA_1$, $A_1 \to bA_2 \mid e$, $A_2 \to c \mid d$ (LL(1)). (b) $S \to \mathtt{i}\,E\,\mathtt{t}\,S\,S' \mid \mathtt{a}$, $S' \to \mathtt{e}\,S \mid \varepsilon$, $E \to \mathtt{b}$. FOLLOW$(S') \supseteq$ FOLLOW$(S) \ni \mathtt{e}$, so $M[S',\mathtt{e}]$ holds both S' productions: not LL(1), because the grammar is ambiguous (dangling else).

Answer: 27. ε-elimination

Nullable: A, B, and S (via AB). $S' \to S \mid \varepsilon$; $S \to AB \mid A \mid B \mid a$; $A \to aA \mid a$; $B \to bB \mid b$. ($S \to a$ and $S \to A$ both give `a`: the language is right; the grammar is now ambiguous, which the question does not forbid.)

Answer: 28. LL(1) or not

(i) yes: FIRST$(aSb) = \{a\}$, FOLLOW$(S) = \{b,\$\}$. (ii) no: common prefix a, both in $M[S,a]$ (factor: $S \to aS'$, $S' \to S \mid \varepsilon$). (iii) no: $M[S,b]$ holds $S \to Aa$ ($b \in$ FIRST(Aa)) and $S \to b$. (iv) no: left-recursive. (v) yes: FIRST$(cA) = \{c\}$, FOLLOW$(A) = \{\$\}$. (vi) no: ambiguous.

Answer: 29. Chomsky classification

(a) Type 3 grammar (right-linear); regular language $(a \mid b)^* a$. (b) Type 2 grammar; $\{a^n b^n\}$ is not regular, so the language is context-free and not regular. (c) Type 2 grammar ($S \to Sb$ is not right-linear), but the language is $a^* b^*$: regular.